RExercise#5_Porras

Renz Benedict Porras

2026-10-21

Part A — Creating Matrices

```
# TODO: Create a 3x3 matrix called m1 containing the numbers 1 to 9,
# using matrix(1:9, nrow = 3, ncol = 3)
# Print m1

m1 <- matrix(1:9, nrow = 3, ncol = 3)

m1
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

```
# TODO: Create a second 3x3 matrix called m2 with the SAME numbers 1:9,
# but this time add the argument byrow = TRUE
# Print m2 and compare it to m1

m2 <- matrix(1:9, nrow = 3, ncol = 3, byrow = TRUE)

m2
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
```

```
# scores <- matrix(c(88, 92, 79, 95, 84, 90), nrow = 2, ncol = 3, byrow = TRUE)
# rownames(scores) <- c("Quiz1", "Quiz2")
# colnames(scores) <- c("Ana", "Ben", "Cid")
# TODO: Print scores and confirm the row/column names were applied

scores <- matrix(c(88, 92, 79, 95, 84, 90), nrow = 2, ncol = 3, byrow = TRUE)

rownames(scores) <- c("Quiz1", "Quiz2")
colnames(scores) <- c("Ana", "Ben", "Cid")

scores
```

```
##       Ana Ben Cid
## Quiz1  88  92  79
## Quiz2  95  84  90
```

Question #1:

```
# Compare m1 and m2 above. Both were built from 1:9 with nrow = 3, ncol = 3, but they look different.
# Explain exactly what byrow = TRUE changes about how R fills a matrix.
m1 <- matrix(1:9, nrow = 3, ncol = 3)
m2 <- matrix(1:9, nrow = 3, ncol = 3, byrow = TRUE)

m1
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

```
m2
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
```

```
# By default, R fills a matrix going down each column first (top to bottom).
# Setting byrow = TRUE tells R to fill the matrix going across each row first (left to right).
```

Question #2:

```
# If you tried matrix(1:8, nrow = 3, ncol = 3) (only 8 numbers for 9 slots), what do you think would ha
# Test it and explain the result, including any warning message.

matrix(1:8, nrow = 3, ncol = 3)
```

```
## Warning in matrix(1:8, nrow = 3, ncol = 3): data length [8] is not a
## sub-multiple or multiple of the number of rows [3]
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    1
```

```
# R fills all 9 slots by reusing (recycling) numbers from the start, so the 9th slot gets the number 1.
# It gives a warning because 8 numbers do not fit into 9 slots evenly.
```

Part B — Subsetting Matrices

```
# Using scores from Part A:
# TODO: Access the single value in row 1, column 2 using scores[1, 2]
# TODO: Access the entire first row using scores[1, ]
# TODO: Access the entire "Ben" column using scores[, "Ben"]

scores <- matrix(c(88, 92, 79, 95, 84, 90), nrow = 2, ncol = 3, byrow = TRUE)
rownames(scores) <- c("Quiz1", "Quiz2")
colnames(scores) <- c("Ana", "Ben", "Cid")

scores[1, 2]
```

```
## [1] 92
```

```r
scores[1, ]
```

```
## Ana Ben Cid
##  88  92  79
```

```r
scores[, "Ben"]
```

```
## Quiz1 Quiz2
##    92    84
```

```r
# TODO: Return only the values in scores that are greater than or equal
# to 90 (hint: scores[scores >= 90] -- note the result loses its shape)

scores[scores >= 90]
```

```
## [1] 95 92 90
```

Question #3:

```r
# scores[1, ] and scores[, "Ben"] both return a result, but one represents a row and the other a column
# Run class() on each result. What data structure does R "downgrade" a single row/column of a matrix to

class(scores[1, ])
```

```
## [1] "numeric"
```

```r
class(scores[, "Ben"])
```

```
## [1] "numeric"
```

```r
# R downgrades a single row or column to a 1D vector (numeric).
# It does this because a single line of values only has one dimension, so R drops the extra matrix dime
```

Question #4:

```r
# In the matrix-logical chunk, the row/column names disappeared from your result.
# Why does subsetting with a logical condition ( scores >= 90 ) lose the 2D shape and names that scores

scores[scores >= 90]
```

```
## [1] 95 92 90
```

```r
# Logical filtering extracts only the specific numbers that match the rule.
# Because these numbers come from separate locations and do not form a complete grid, R flattens the re
```

Part C — Matrix Properties and Operations

```r
# TODO: Use dim(), nrow(), and ncol() on scores to report its dimensions

dim(scores)
```

```
## [1] 2 3
```

```r
nrow(scores)
```

```
## [1] 2
```

```r
ncol(scores)
```

```
## [1] 3
```

```r
# TODO: Use t() to transpose scores and store it as scores_t
# Print scores_t and compare its dimensions to scores using dim()

scores_t <- t(scores)

scores_t
```

```
##     Quiz1 Quiz2
## Ana    88    95
## Ben    92    84
## Cid    79    90
```

```r
dim(scores_t)
```

```
## [1] 3 2
```

```r
dim(scores)
```

```
## [1] 2 3
```

```r
# a <- matrix(1:4, nrow = 2)
# b <- matrix(5:8, nrow = 2)
# TODO: Add a and b element-wise (a + b)
# TODO: Multiply a and b element-wise (a * b)

a <- matrix(1:4, nrow = 2)
b <- matrix(5:8, nrow = 2)

a + b
```

```
##      [,1] [,2]
## [1,]    6   10
## [2,]    8   12
```

```r
a * b
```

```
##      [,1] [,2]
## [1,]    5   21
## [2,]   12   32
```

```r
# TODO: Perform true matrix multiplication of a and b using %*%
# Compare this result to a * b above -- they should be different

a %*% b
```

```
##      [,1] [,2]
## [1,]   23   31
## [2,]   34   46
```

Question #5:

```r
# a * b and a %*% b both "multiply" the same two matrices but give different results.
# Explain the difference between element-wise multiplication and matrix multiplication in your own words
```

```r
a <- matrix(1:4, nrow = 2)
b <- matrix(5:8, nrow = 2)

a * b
```

```
##      [,1] [,2]
## [1,]    5   21
## [2,]   12   32
```

```r
a %*% b
```

```
##      [,1] [,2]
## [1,]   23   31
## [2,]   34   46
```

```r
# a * b (element-wise) simply multiplies numbers that share the exact same spot in both matrices.
# a %*% b (matrix multiplication) multiplies each row of the first matrix by each column of the second
```

Question #6:

```r
# t(scores) swaps rows and columns. Look at dim(scores) versus dim(scores_t).
# Why must the row/column names also swap positions when you transpose a matrix?
scores <- matrix(c(88, 92, 79, 95, 84, 90), nrow = 2, ncol = 3, byrow = TRUE)
rownames(scores) <- c("Quiz1", "Quiz2")
colnames(scores) <- c("Ana", "Ben", "Cid")
scores_t <- t(scores)

dim(scores)
```

```
## [1] 2 3
```

```r
dim(scores_t)
```

```
## [1] 3 2
```

```r
# Row and column names swap so that every label stays attached to its original values when rows turn in
```

Part D — cbind() and rbind()

```r
# TODO: A new student "Dex" scored 85 on Quiz1 and 91 on Quiz2.
# Use cbind() to add a new column to scores called Dex with
# these two values. Store the result as scores_v2 and print it.
scores_v2 <- cbind(scores, Dex = c(85, 91))

scores_v2
```

```
##       Ana Ben Cid Dex
## Quiz1  88  92  79  85
## Quiz2  95  84  90  91
```

```r
# TODO: A new quiz "Quiz3" was given with scores 80, 88, 93 for
# Ana, Ben, Cid (use only the original 3 students, not Dex).
# Use rbind() to add this as a new row to scores. Store as scores_v3
# and print it.
scores_v3 <- rbind(scores, Quiz3 = c(80, 88, 93))
```

```
scores_v3
```

```
##       Ana Ben Cid
## Quiz1  88  92  79
## Quiz2  95  84  90
## Quiz3  80  88  93
```

Question #7:

```
# What would happen if you tried cbind(scores, c(85, 91, 100)) - giving 3 values for a matrix that only
# Test it, read the warning/error, and explain why cbind() and rbind() require matching dimensions.
scores <- matrix(c(88, 92, 79, 95, 84, 90), nrow = 2, ncol = 3, byrow = TRUE)
rownames(scores) <- c("Quiz1", "Quiz2")
colnames(scores) <- c("Ana", "Ben", "Cid")

cbind(scores, c(85, 91, 100))
```

```
## Warning in cbind(scores, c(85, 91, 100)): number of rows of result is not a
## multiple of vector length (arg 2)
```

```
##       Ana Ben Cid
## Quiz1  88  92  79 85
## Quiz2  95  84  90 91
```

```
# R gives a warning because 3 values do not fit cleanly into a 2-row matrix.
# cbind() and rbind() need matching lengths so every row and column has the exact same size, keeping th
```

Question #8:

```
# In what situation would you use rbind() instead of cbind() when updating a dataset over time?
# Give a concrete example (e.g., daily sales, weekly attendance) and explain your reasoning.

# You use rbind() when adding new data entries over time (rows), instead of adding new categories or va
# Example: In tracking daily sales, each new day creates a new row with date and total sales. Using rbi
```

Part E — Arrays: Creating and Subsetting

```
# TODO: Create a 2x3x2 array called arr using:
# array(1:12, dim = c(2, 3, 2))
# Print arr and observe how it displays as two separate 2x3 "slices"
arr <- array(1:12, dim = c(2, 3, 2))

arr
```

```
## , , 1
##
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
##
## , , 2
##
```

```
##      [,1] [,2] [,3]
## [1,]    7    9   11
## [2,]    8   10   12
```

```
# TODO: Access the element in row 1, column 2, of the SECOND Layer
# (slice) of arr, e.g. arr[1, 2, 2]

arr[1, 2, 2]
```

```
## [1] 9
```

```
# TODO: Access the entire first layer (slice) of arr using arr[, , 1]

arr[, , 1]
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
# TODO: Use dim() on arr to confirm its dimensions

dim(arr)
```

```
## [1] 2 3 2
```

Question #9:

```
# A matrix needs 2 indices ( scores[row, col] ) but an array needs 3 ( arr[row, col, layer] ).
# Using dim(arr), explain in your own words what each of the three numbers represents.

arr <- array(1:12, dim = c(2, 3, 2))
dim(arr)
```

```
## [1] 2 3 2
```

```
# The three numbers returned by dim(arr) represent:
# 1. 2 = Number of rows in each table/slice.
# 2. 3 = Number of columns in each table/slice.
# 3. 2 = Number of layers (slices) stacked together in the 3D structure.
```

Question #10:

```
# Give a real-world example of data that would naturally fit a 3D array structure
# (hint: think of something that has rows, columns, AND a third repeating dimension like time, subjects
# Explain why a single 2D matrix would not be enough.

# Example: Monthly sales of store products.
# Rows: Product items (e.g., Laptops, Phones, Tablets)
# Columns: Store locations (e.g., North Branch, South Branch, East Branch)
# Layers (3rd dimension): Months of the year (e.g., Jan, Feb, Mar...)

# Why a single 2D matrix is not enough:
# A 2D matrix can only show two pieces of information at once (products vs. store locations for a singl
# To track sales for multiple months without mixing or duplicating tables, a third dimension is needed
```

Part F — Critical Thinking Challenge

```
# class_a <- matrix(c(85, 90, 78, 92), nrow = 2, byrow = TRUE,
#                   dimnames = list(c("S1", "S2"), c("Midterm", "Final")))
# class_b <- matrix(c(88, 84, 95, 91), nrow = 2, byrow = TRUE,
#                   dimnames = list(c("S3", "S4"), c("Midterm", "Final")))

class_a <- matrix(c(85, 90, 78, 92), nrow = 2, byrow = TRUE,
                  dimnames = list(c("S1", "S2"), c("Midterm", "Final")))
class_b <- matrix(c(88, 84, 95, 91), nrow = 2, byrow = TRUE,
                  dimnames = list(c("S3", "S4"), c("Midterm", "Final")))

# TODO: Combine class_a and class_b into one matrix called all_students
# using the correct bind function (think about whether these represent
# new STUDENTS or new SCORE COLUMNS before choosing cbind or rbind)
all_students <- rbind(class_a, class_b)

all_students
```

```
##    Midterm Final
## S1      85    90
## S2      78    92
## S3      88    84
## S4      95    91
```

```
# TODO: From all_students, compute each student's average score across
# Midterm and Final. Hint: rowMeans(all_students) does this in one line.

rowMeans(all_students)
```

```
##   S1   S2   S3   S4
## 87.5 85.0 86.0 93.0
```

Question #11:

```
# Explain WHY you chose cbind() or rbind() in challenge-1.
# What clue in the data (shared row names vs. shared column names) told you which function was correct?

# I chose rbind() because we were adding new student rows to the dataset.
# The clue in the data was the shared column names ("Midterm" and "Final"). Since both matrices shared
```

Question #11:

```
# Question #12:
# Suppose instead of 2 classes you had 5 classes, each recorded as a separate matrix with the same 2 co
# Describe, in plain language (no code required), how you would combine all 5 into a single matrix, and

# To combine all 5 matrices, I would stack them vertically line-by-line using rbind() with all 5 matrix
# To check the result with dim(), the output should show 2 columns (for Midterm and Final) and a row co
```

CONCLUSION:

```
# In 3-4 sentences, summarize the key differences between a matrix and an array, and describe a situati

# A matrix is a 2D grid containing rows and columns, while an array extends this into 3 or more dimensi
```